

Report

Laporan Final Project Sistem Basis Data 2026

PokeZOO Management System: Sistem Manajemen Kebun Binatang
Pokémon



Oleh: Kelompok 1:

- Umar (5027251005)
- Gede Satya Putra Aryanta (5027251012)
- Muhammad Syadzili Abdul Muhyi (5027251030)
- Catur Setyo Ragil (5027251066)

Kelas : B

1. Latar Belakang

PokeZOO Management System adalah sistem manajemen kebun binatang Pokemon yang dirancang untuk mengelola data operasional seperti Pokemon, habitat, keeper, feeding schedule, ticketing, interaksi pengunjung, behavior log, incident report, dan visitor review.

Sistem ini menggunakan pendekatan hybrid database karena data yang dikelola memiliki karakteristik berbeda. Data operasional utama bersifat terstruktur dan saling berelasi, sedangkan data seperti log, laporan insiden, dan review pengunjung lebih fleksibel dan cocok disimpan dalam bentuk dokumen.

2. Identifikasi Masalah

Permasalahan yang diangkat dalam project ini adalah:

- Pengelolaan data zoo memiliki banyak entitas yang saling berhubungan.
- Integritas relasional pada data transaksi, termasuk pengelolaan tiket, jadwal pemberian makan, serta penugasan petugas (keeper).
- Data dokumentasi seperti behavior log, incident report, dan review memiliki struktur yang lebih fleksibel.
- Sistem membutuhkan pembagian akses berdasarkan role admin, keeper, dan visitor

3. Fungsionalitas Sistem

Sistem memiliki tiga role utama:

Admin:

- Mengelola Pokemon, species, habitat, keeper, food, dan feeding schedule.
- Melihat dashboard statistik.
- Melihat health lifecycle Pokemon.
- Mengakses SQL Playground, MongoDB Viewer, MongoDB Playground, dan Reports.

Keeper:

- Melihat Pokemon yang ditugaskan.
- Mengupdate status feeding schedule.
- Membuat behavior log.
- Membuat incident report.

Visitor:

- Membeli tiket.
- Melihat habitat dan Pokemon.
- Melakukan interaksi dengan Pokemon.
- Memberikan review.

4. Desain Relational Database

Data relasional pada PokeZOO Management System disimpan menggunakan MySQL/MariaDB. Database relasional digunakan untuk data yang memiliki hubungan antar entitas yang jelas.

Entitas utama yang digunakan pada database relasional adalah:

1. users

Tabel users menyimpan data akun pengguna sistem. Atribut utamanya adalah user_id, username, password, dan role. Role digunakan untuk membedakan hak akses admin, keeper, dan visitor.

2. keepers

Tabel keepers menyimpan data petugas zoo. Atribut utamanya adalah keeper_id, user_id, name, shift, dan phone_number. Tabel ini berelasi dengan users melalui user_id.

3. visitors

Tabel visitors menyimpan data pengunjung. Atribut utamanya adalah visitor_id, user_id, name, email, dan phone_number. Tabel ini berelasi dengan users melalui user_id.

4. pokemon_species

Tabel pokemon_species menyimpan data spesies Pokemon. Atribut utamanya adalah species_id, species_name, dan rarity.

5. pokemon_types

Tabel pokemon_types menyimpan daftar tipe Pokemon. Atribut utamanya adalah type_id dan type_name.

6. species_type

Tabel species_type merupakan tabel penghubung many-to-many antara pokemon_species dan pokemon_types. Satu species dapat memiliki lebih dari satu type, dan satu type dapat dimiliki oleh banyak species.

7. habitats

Tabel habitats menyimpan data habitat Pokemon. Atribut utamanya adalah habitat_id, habitat_name, habitat_type, capacity, dan status.

8. pokemon

Tabel pokemon menyimpan data Pokemon yang ada di zoo. Atribut utamanya adalah `pokemon_id`, `species_id`, `habitat_id`, `nickname`, `health_status`, `status`, dan `entry_date`. Tabel ini berelasi dengan `pokemon_species` dan `habitats`.

9. pokemon_keepers

Tabel `pokemon_keepers` merupakan tabel penghubung many-to-many antara pokemon dan keepers. Satu Pokemon dapat ditangani oleh beberapa keeper, dan satu keeper dapat menangani beberapa Pokemon.

10. foods

Tabel `foods` menyimpan data makanan Pokemon. Atribut utamanya adalah `food_id`, `food_name`, `nutrition`, dan `stock`.

11. feeding_schedules

Tabel `feeding_schedules` menyimpan jadwal pemberian makan Pokemon. Atribut utamanya adalah `feeding_id`, `pokemon_id`, `keeper_id`, `food_id`, `feeding_time`, dan `status`. Tabel ini berelasi dengan `pokemon`, `keepers`, dan `foods`.

12. tickets

Tabel `tickets` menyimpan data tiket pengunjung. Atribut utamanya adalah `ticket_id`, `visitor_id`, `visit_date`, `ticket_type`, `payment_method`, `purchase_date`, `price`, dan `status`. Tabel ini berelasi dengan `visitors`.

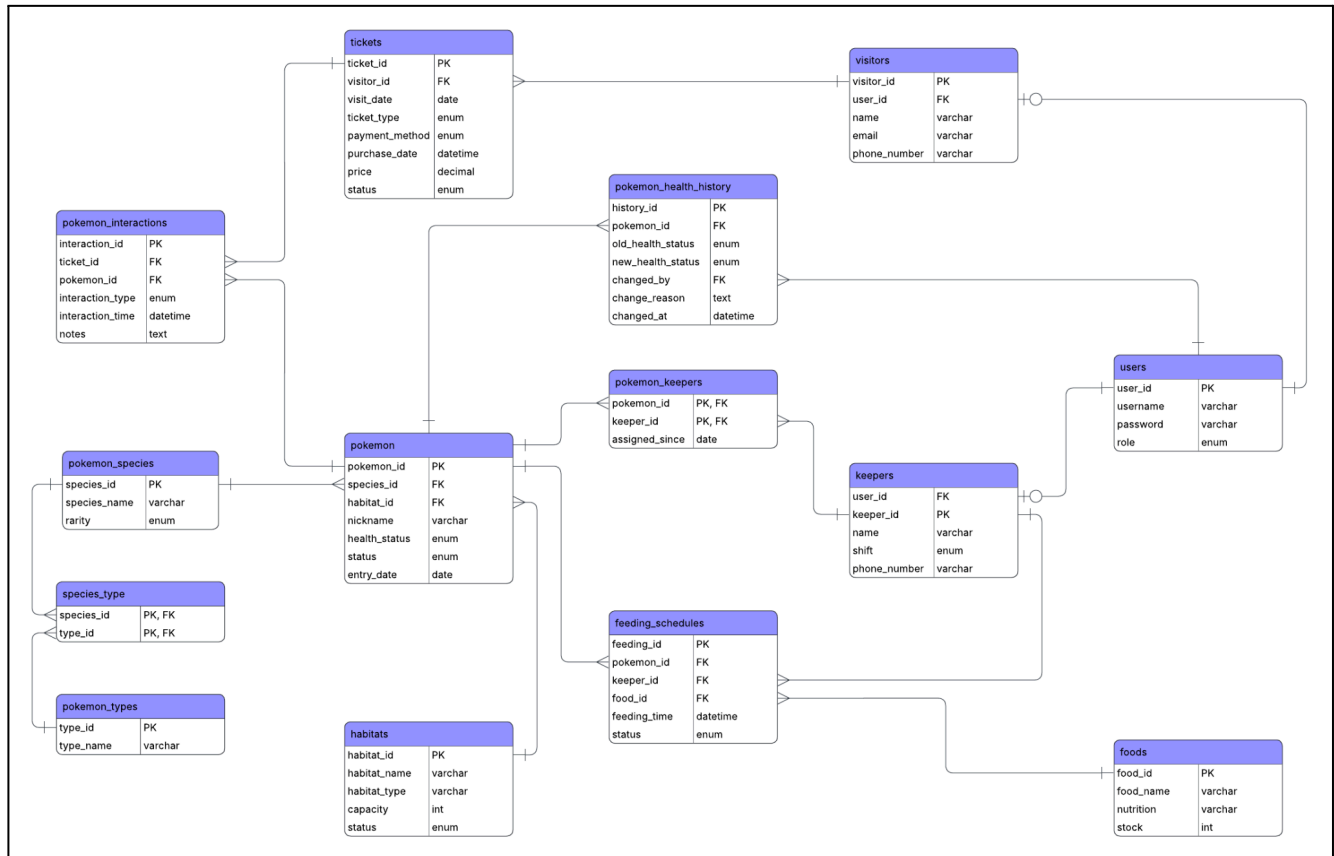
13. pokemon_interactions

Tabel `pokemon_interactions` menyimpan data interaksi pengunjung dengan Pokemon. Atribut utamanya adalah `interaction_id`, `ticket_id`, `pokemon_id`, `interaction_type`, `interaction_time`, dan `notes`. Tabel ini berelasi dengan `tickets` dan `pokemon`.

14. pokemon_health_history

Tabel `pokemon_health_history` menyimpan riwayat perubahan status kesehatan Pokemon. Atribut utamanya adalah `history_id`, `pokemon_id`, `old_health_status`, `new_health_status`, `changed_by`, `change_reason`, dan `changed_at`. Tabel ini berelasi dengan `pokemon` dan `users`.

5. ERD



6. Pembagian Data Relational dan Non-Relational

MySQL/MariaDB digunakan untuk:

- Data user dan role.
- Data Pokemon, species, type, dan habitat.
- Data keeper dan assignment.
- Data food dan feeding schedule.
- Data visitor, ticket, dan interaction.
- Data health lifecycle Pokemon.

MongoDB digunakan untuk:

- Behavior logs.
- Incident reports.
- Visitor reviews.

Alasan pembagian:

- MySQL cocok untuk data yang membutuhkan relasi kuat dan transaksi.
- MongoDB cocok untuk data semi-structured yang field-nya dapat berkembang.

7. Desain Collection MongoDB

Selain menggunakan MySQL/MariaDB untuk data relasional, PokeZOO Management System juga menggunakan MongoDB untuk menyimpan data yang bersifat lebih fleksibel dan semi-terstruktur. Data yang disimpan pada MongoDB merupakan data dokumentatif, seperti behavior log, incident report, dan visitor review.

MongoDB dipilih karena dapat menyimpan data dalam bentuk dokumen sehingga lebih fleksibel apabila terdapat field tambahan pada masa pengembangan sistem. Pada sistem PokeZOO, MongoDB digunakan untuk tiga collection utama, yaitu *pokemon_behavior_logs*, *incident_reports*, dan *visitor_reviews*.

7.1. *Pokemon_behavior_logs*

Collection *pokemon_behavior_logs* digunakan untuk menyimpan catatan perilaku Pokemon yang dibuat oleh keeper. Data ini berisi hasil observasi keeper terhadap kondisi, aktivitas, mood, atau respons Pokemon pada waktu tertentu.

Collection ini cocok disimpan menggunakan MongoDB karena catatan perilaku Pokemon dapat memiliki detail yang berbeda-beda. Misalnya, pada beberapa kondisi keeper dapat menambahkan informasi seperti cuaca, penyebab perubahan perilaku, atau item enrichment yang digunakan.

Cakupan utama pada collection ini meliputi *pokemon_id*, *keeper_user_id*, *date*, *time*, *behavior*, dan *mood*.

```
db.pokemon_behavior_logs.insertMany([
  {
    pokemon_id: 1,
    keeper_user_id: 3,
    date: "2026-06-15",
    time: "08:20:00",
    behavior: "Sparky responded quickly to feeding cues and interacted calmly with visitors.",
    mood: "happy",
    trigger: "Morning feeding session",
    weather: "Clear",
    enrichment_item: "Electric-safe puzzle feeder"
  },
  {
    pokemon_id: 3,
    keeper_user_id: 3,
    date: "2026-06-15",
    time: "10:45:00",
    behavior: "Blaze practiced controlled ember bursts during the keeper-supervised show.",
    mood: "calm",
    trigger: "Scheduled fire show",
    flame_intensity: "low"
  },
  {
    pokemon_id: 8,
    keeper_user_id: 6,
    date: "2026-06-15",
    time: "11:30:00",
    behavior: "Nova stayed near shaded plants and showed reduced appetite.",
    mood: "lethargic",
    trigger: "Post-health observation",
    appetite_level: "low",
    follow_up_required: true
  }
])
```

Gambar 7.1 Contoh ampilan collection *pokemon_behavior_logs* pada MongoDB Viewer.

7.2. *Incident_reports*

Collection *incident_reports* digunakan untuk menyimpan laporan insiden yang terjadi pada Pokemon atau habitat. Laporan ini dibuat oleh keeper ketika terjadi kejadian tertentu yang membutuhkan pencatatan, tindakan, atau tindak lanjut.

Data incident report disimpan di MongoDB karena setiap laporan insiden dapat memiliki detail yang berbeda-beda. Beberapa laporan mungkin hanya berisi deskripsi dan tingkat keparahan, sedangkan laporan lain dapat memiliki daftar tindakan, status tindak lanjut, atau penyebab insiden.

Cakupan utama pada collection ini meliputi *incident_id*, *keeper_user_id*, *date_reported*, *pokemon_id*, *habitat_id*, *severity*, *description*, dan *actions_taken*.

```
db.incident_reports.insertMany([
  {
    incident_id: "INC-DEMO-001",
    keeper_user_id: 4,
    date_reported: "2026-06-13T12:20:00.000Z",
    pokemon_id: 4,
    habitat_id: 2,
    severity: "Medium",
    description: "Ember slipped near the warm stone area and showed minor tail discomfort.",
    actions_taken: ["moved to resting zone", "applied cooling treatment", "notified admin"],
    follow_up_status: "monitor for 24 hours"
  },
  {
    incident_id: "INC-DEMO-002",
    keeper_user_id: 5,
    date_reported: "2026-06-14T21:30:00.000Z",
    pokemon_id: 11,
    habitat_id: 5,
    severity: "High",
    description: "Shadow displayed abnormal behavior and startled nearby Pokemon in Crystal Cave.",
    actions_taken: ["secured cave section", "moved visitors away", "started quarantine protocol"],
    reported_area: "Crystal Cave north corridor"
  },
  {
    incident_id: "INC-DEMO-003",
    keeper_user_id: 6,
    date_reported: "2026-06-15T09:40:00.000Z",
    pokemon_id: 8,
    habitat_id: 4,
    severity: "Low",
    description: "Nova showed low appetite during morning observation.",
    actions_taken: ["recorded symptoms", "prepared lighter meal", "scheduled health recheck"],
    suspected_cause: "temporary illness"
  }
])
```

Gambar 7.2 Contoh Tampilan collection *incident_reports* pada MongoDB Viewer.

7.3. *Visitor_reviews*

Collection *visitor_reviews* digunakan untuk menyimpan review yang diberikan oleh pengunjung. Review ini dapat berupa rating, komentar, habitat favorit, dan waktu review diberikan.

Data review cocok disimpan di MongoDB karena komentar pengunjung bersifat fleksibel. Pengunjung dapat memberikan komentar dengan panjang dan isi yang berbeda-beda, sehingga penyimpanan berbasis dokumen lebih sesuai untuk jenis data ini.

Field utama pada collection ini meliputi *visitor_id*, *rating*, *comment*, *favorite_habitat*, dan *date_submitted*.

```
db.visitor_reviews.insertMany([
  {
    visitor_id: 1,
    rating: 5,
    comment: "The VIP pass was worth it. Sparky and Blaze were the highlights of the visit.",
    favorite_habitat: "Thunder Meadow",
    date_submitted: "2026-06-15T17:30:00.000Z"
  },
  {
    visitor_id: 2,
    rating: 4,
    comment: "The show was fun and the habitat layout was easy to follow.",
    favorite_habitat: "Mystic Forest",
    date_submitted: "2026-06-15T18:10:00.000Z"
  },
  {
    visitor_id: 3,
    rating: 5,
    comment: "Great educational visit. I liked seeing the ticket interaction system in action.",
    favorite_habitat: "Aqua Lagoon",
    date_submitted: "2026-06-16T12:00:00.000Z"
  }
])
```

Gambar 7.3 Contoh tampilan collection *visitor_reviews* pada MongoDB Viewer.

8. Embedding dan Referencing

Pada MongoDB, sistem PokeZOO lebih banyak menggunakan pendekatan referencing. Field seperti *pokemon_id*, *visitor_id*, *habitat_id*, dan *keeper_user_id* digunakan sebagai referensi ke data utama yang tersimpan pada MySQL/MariaDB.

Contohnya, pada collection *incident_reports*, sistem menyimpan *pokemon_id* dan *habitat_id*, bukan menyimpan seluruh detail Pokemon dan habitat secara langsung di dalam dokumen MongoDB. Ketika sistem perlu menampilkan nama Pokemon atau nama habitat, backend dapat mengambil data detail tersebut dari database relasional berdasarkan ID yang tersimpan.

Pendekatan referencing dipilih karena data utama seperti Pokemon, keeper, visitor, dan habitat sudah memiliki struktur yang jelas di MySQL/MariaDB. Dengan cara ini, sistem dapat mengurangi duplikasi data dan menjaga konsistensi data utama.

Sementara itu, embedding digunakan untuk data yang memang menjadi bagian langsung dari satu dokumen. Contohnya adalah field *actions_taken* pada collection *incident_reports*. Field tersebut disimpan sebagai array karena daftar tindakan merupakan bagian dari laporan insiden itu sendiri.

Alasan desain embedding dan referencing:

1. Mengurangi duplikasi data.
2. Menjaga data utama tetap konsisten di MySQL/MariaDB.
3. Memungkinkan MongoDB tetap fleksibel untuk field tambahan.
4. Memisahkan data utama yang relasional dengan data dokumentatif yang semi-terstruktur.

9. Implementasi Backend

Backend PokeZOO Management System dibangun menggunakan FastAPI. Aplikasi ini menggunakan pendekatan server-side rendering dengan Jinja2 templates, sehingga halaman HTML dirender langsung dari backend.

Struktur backend dipisahkan berdasarkan role pengguna, yaitu admin, keeper, dan visitor. Pembagian ini membuat sistem lebih mudah dikembangkan karena setiap role memiliki route dan fitur masing-masing.

Teknologi yang digunakan:

1. FastAPI sebagai backend framework.
2. PyMySQL untuk koneksi ke MySQL/MariaDB.
3. Motor untuk koneksi asynchronous ke MongoDB.
4. Jinja2 untuk template HTML.
5. TailwindCSS melalui CDN untuk styling tampilan.
6. Session-based authentication untuk autentikasi user.
7. Role-based access control untuk membatasi akses berdasarkan role.

Secara umum, struktur backend terdiri dari:

- | | |
|---------------------------|--|
| 1. <i>main.py</i> | sebagai entry point aplikasi. |
| 2. <i>database.py</i> | untuk konfigurasi koneksi database |
| 3. <i>routes/auth.py</i> | untuk login, logout, dan register visitor. |
| 4. <i>routes/admin/</i> | untuk fitur admin. |
| 5. <i>routes/keeper/</i> | untuk fitur keeper. |
| 6. <i>routes/visitor/</i> | untuk fitur visitor. |
| 7. <i>templates/</i> | untuk file tampilan HTML. |

Sistem autentikasi menggunakan session. Setelah user berhasil login, informasi user dan role akan disimpan pada session. Role tersebut kemudian digunakan untuk menentukan halaman yang boleh diakses oleh user.

Admin dapat mengelola data utama seperti Pokemon, species, habitat, keeper, food, dan feeding schedule. Admin juga dapat mengakses dashboard, SQL Playground, MongoDB Viewer, dan Reports.

Keeper dapat melihat Pokemon yang ditugaskan, mengupdate feeding schedule, membuat behavior log, dan membuat incident report.

Visitor dapat membeli tiket, melihat habitat dan Pokemon, melakukan interaksi dengan Pokemon, serta memberikan review.

10. Fitur Database

Fitur database yang diterapkan:

1. MySQL/MariaDB digunakan untuk menyimpan data relasional utama seperti user, Pokemon, habitat, keeper, feeding schedule, ticketing, dan health history.
2. MongoDB digunakan untuk menyimpan data semi-structured seperti behavior logs, incident reports, dan visitor reviews.
3. Foreign key digunakan untuk menjaga integritas data antar tabel.
4. Relasi many-to-many diterapkan pada species_type dan pokemon_keepers.
5. Trigger digunakan untuk validasi stok makanan dan pengurangan stok otomatis setelah feeding selesai.
6. Transaction digunakan pada operasi multi-step agar data tetap konsisten, seperti penambahan Pokemon, species, dan keeper.
7. Health lifecycle dicatat melalui tabel pokemon_health_history.
8. MongoDB menggunakan referencing melalui pokemon_id, visitor_id, habitat_id, dan keeper_user_id untuk menghubungkan dokumen dengan data MySQL.
9. MongoDB aggregation digunakan untuk admin reports.
10. SQL Playground dan MongoDB Viewer/Playground digunakan untuk mengetes query langsung dari aplikasi.

11. Kesimpulan

PokeZOO Management System menerapkan konsep hybrid database dengan menggunakan MySQL/MariaDB untuk data relasional dan MongoDB untuk data semi-terstruktur. MySQL/MariaDB digunakan untuk menjaga integritas data utama seperti user, Pokemon, habitat, keeper, feeding schedule, ticketing, dan health history, sedangkan MongoDB digunakan untuk menyimpan behavior logs, incident reports, dan visitor reviews yang lebih fleksibel.

Dengan pembagian role admin, keeper, dan visitor, sistem ini mampu mendukung kebutuhan pengelolaan data zoo, operasional keeper, serta interaksi pengunjung. Secara keseluruhan, PokeZOO Management System menunjukkan penerapan basis data relasional dan non-relasional dalam satu sistem yang saling terintegrasi.

TO DO

PokeZOO Management System

Final Project Sistem Basis Data 2026

1. Identitas Kelompok

Nama anggota:

- ...
- ...
- ...

Kelas:

...

2. Latar Belakang

PokeZOO Management System adalah sistem manajemen kebun binatang Pokemon yang dirancang untuk mengelola data operasional seperti Pokemon, habitat, keeper, feeding schedule, ticketing, interaksi pengunjung, behavior log, incident report, dan visitor review.

Sistem ini menggunakan pendekatan hybrid database karena data yang dikelola memiliki karakteristik berbeda. Data operasional utama bersifat terstruktur dan saling berelasi, sedangkan data seperti log, laporan insiden, dan review pengunjung lebih fleksibel dan cocok disimpan dalam bentuk dokumen.

3. Identifikasi Masalah

Permasalahan yang diangkat dalam project ini adalah:

- Pengelolaan data zoo memiliki banyak entitas yang saling berhubungan.
- Data transaksi seperti ticketing, feeding schedule, dan assignment keeper membutuhkan integritas relasional.
- Data dokumentatif seperti behavior log, incident report, dan review memiliki struktur yang lebih fleksibel.
- Sistem membutuhkan pembagian akses berdasarkan role admin, keeper, dan visitor.

4. Fungsionalitas Sistem

Sistem memiliki tiga role utama:

Admin:

- Mengelola Pokemon, species, habitat, keeper, food, dan feeding schedule.
- Melihat dashboard statistik.
- Melihat health lifecycle Pokemon.
- Mengakses SQL Playground, MongoDB Viewer, MongoDB Playground, dan Reports.

Keeper:

- Melihat Pokemon yang ditugaskan.
- Mengupdate status feeding schedule.
- Membuat behavior log.

- Membuat incident report.

Visitor:

- Membeli tiket.
- Melihat habitat dan Pokemon.
- Melakukan interaksi dengan Pokemon.
- Memberikan review.

5. Desain Relational Database

Data relasional disimpan menggunakan MySQL/MariaDB karena membutuhkan struktur tabel, relasi, foreign key, dan integritas data.

Entitas utama:

- users
- keepers
- visitors
- pokemon_species
- pokemon_types
- species_type
- habitats
- pokemon
- pokemon_keepers
- foods
- feeding_schedules
- tickets
- pokemon_interactions
- pokemon_health_history

Relasi penting:

- users memiliki relasi dengan keepers atau visitors.
- pokemon_species berelasi many-to-many dengan pokemon_types melalui species_type.
- pokemon berelasi dengan habitats.
- pokemon berelasi many-to-many dengan keepers melalui pokemon_keepers.
- visitors memiliki banyak tickets.
- tickets memiliki banyak pokemon_interactions.
- pokemon memiliki banyak pokemon_health_history.

6. ERD

[Masukkan gambar ERD di sini]

ERD digunakan untuk menggambarkan struktur data relasional pada sistem PokeZOO. Desain ERD memastikan setiap data operasional memiliki hubungan yang jelas dan mendukung integritas data melalui primary key dan foreign key.

7. Pembagian Data Relational dan Non-Relational

MySQL/MariaDB digunakan untuk:

- Data user dan role.
- Data Pokemon, species, type, dan habitat.
- Data keeper dan assignment.
- Data food dan feeding schedule.
- Data visitor, ticket, dan interaction.
- Data health lifecycle Pokemon.

MongoDB digunakan untuk:

- Behavior logs.
- Incident reports.
- Visitor reviews.

Alasan pembagian:

- MySQL cocok untuk data yang membutuhkan relasi kuat dan transaksi.
- MongoDB cocok untuk data semi-structured yang field-nya dapat berkembang.

8. Desain Collection MongoDB

Sistem menggunakan tiga collection utama:

8.1 pokemon_behavior_logs

Digunakan untuk menyimpan catatan perilaku Pokemon dari keeper.

Contoh field:

- pokemon_id
- keeper_user_id
- date
- time
- behavior
- mood
- trigger
- dynamic/custom fields

8.2 incident_reports

Digunakan untuk menyimpan laporan insiden pada habitat atau Pokemon.

Contoh field:

- incident_id
- keeper_user_id
- date_reported
- pokemon_id
- habitat_id
- severity

- description
- actions_taken
- dynamic/custom fields

8.3 visitor_reviews

Digunakan untuk menyimpan review pengunjung.

Contoh field:

- visitor_id
- rating
- comment
- favorite_habitat
- date_submitted

9. Embedding dan Referencing

Pada MongoDB, sistem lebih banyak menggunakan pendekatan referencing. Field seperti pokemon_id, visitor_id, habitat_id, dan keeper_user_id digunakan untuk menghubungkan dokumen MongoDB dengan data utama yang tersimpan di MySQL.

Embedding digunakan untuk data yang masih menjadi bagian alami dari satu dokumen, seperti actions_taken pada incident_reports. Data tersebut disimpan sebagai array karena merupakan bagian langsung dari laporan insiden.

Alasan desain:

- Mengurangi duplikasi data.
- Menjaga data utama tetap konsisten di MySQL.
- Memungkinkan MongoDB tetap fleksibel untuk field tambahan.

10. Implementasi Backend

Backend dibangun menggunakan FastAPI. Aplikasi menggunakan server-side rendering dengan Jinja2 templates.

Teknologi:

- FastAPI sebagai backend framework.
- PyMySQL untuk koneksi MySQL/MariaDB.
- Motor untuk koneksi MongoDB.
- Jinja2 untuk template HTML.
- Session-based authentication.
- Role-based access control.

11. Fitur Database

Fitur database yang diterapkan:

- Foreign key untuk menjaga integritas data.
- Trigger untuk validasi stok makanan pada feeding schedule.

- Trigger untuk mengurangi stok makanan setelah feeding selesai.
- Transaction untuk operasi multi-step seperti insert Pokemon dan assignment keeper.
- Health lifecycle menggunakan tabel pokemon_health_history.
- MongoDB aggregation untuk admin reports.

12. Kesimpulan

PokeZOO Management System menerapkan hybrid database untuk memisahkan data berdasarkan karakteristiknya. MySQL/MariaDB digunakan untuk data relasional yang membutuhkan integritas dan transaksi, sedangkan MongoDB digunakan untuk data fleksibel seperti behavior log, incident report, dan visitor review.

Dengan desain ini, sistem dapat mendukung kebutuhan admin, keeper, dan visitor secara terstruktur, konsisten, dan fleksibel.